Python-workshop

Välkommen till vår Python-workshop! Denna workshop är uppdelad i olika svårighetsgrader för att matcha olika personers tidigare erfarenhet av programmering/Python.

Delarna är uppdelade enligt:

A. Introduktion
B. Start för nybörjare till Python/programmering
C. Problem för dig som programmerat innan (Python)
D. Problem för dig som är mer erfaren

Man får även självklart göra något helt eget projekt under workshoparna (behöver inte ens vara Python). Det är även upp till er om ni arbetar i par eller grupp, ni kommer nog behöva dela på datorerna för att de ska räcka till. Lycka till och tveka inte att fråga om hjälp från oss!

Kapitel A.1, A.2, A.3 kan vara bra att läsa igenom även om man programmerat i Python innan då det går igenom hur man använder LiUs datorer.

orkade inte uppdatera innehållsförteckning lol //luttu

Innehållsförteckning

A. Komma igång med programvara **3**
1. Öppna en terminal 3
2. Ladda programmet VS Code 3
3. Skapa en fil och öppna i text-editor

B. Komma igång med Python **4**
1. Skapa och starta ett program 4
Syntax i Python 5
Variabler 5
If-satser 5
Funktioner 6
Loopar 6

C. Medelsvåra uppgifter **7**
1. Summera alla jämna tal från 0-99 7
2. Hitta största i en lista av siffror 7
3. Skriv ett program som skapar och skriver text till en fil 7
4. Skriv en rekursiv funktion som räknar ut fakulteten av ett tal 8
5. Skriv en funktion som genererar Fibonaccis talserie 8

D. Svåra uppgifter **9**
1. Skapa en funktion som översätter morsekod till bokstäver 9
Fler svårare problem 11

Lösningsförslag **12**

A. Komma igång med LiUs datorer

Följ stegen nedan för att komma igång med att använda dessa datorer.

1. Öppna en terminal

Öppna en terminal genom att klicka på terminal-fönstret i vänstra hörnet eller med shortcut CTRL+ALT+T.

2. Ladda programmet VS Code

För att kunna använda text-editorn VS Code på LiUs datorer måste man först ladda in programmet. Detta görs genom följande kommando, skriv in det i terminalen du precis öppnade:

```
None
module add prog/vscode
```

(detta behövs göras varje gång du loggar in på datorn)

3. Skapa en fil och öppna i text-editor

Starta VS Code och skapa en ny Python-fil i terminalen genom att skriva in följande:

```
None
code start.py
```

(detta skapar en ny fil kallad “start.py” i din home-folder)

B. Syntax

Syntax beskriver hur man skriver kod, och varierar väldigt mycket mellan olika programmeringsspråk.T.ex att skriva ut text i C++ ser ut såhär:

```C/C++
#include <iostream>

int main() {
       std::cout << "Hello, World!" << std::endl;
       return 0;
}
```

och i java såhär:

```Java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Dessa två språk är statiskt typade, vilket bland annat innebär att man måste säga åt programmet vad man tänker stoppa in för typ av värde i varje variabel och vad funktioner (mer om funktioner lite längre ner i dokumentet) ska returnera när de är färdiga.

Python är dock ett dynamiskt typat språk, vilket innebär att man kan göra lite vafan man vill, vilket i sin tur innebär att det är ett lite lättare programmeringsspråk att börja lära sig, och i sin tur anledningen till att vi kommer använda oss utav det idag.

Detta är inte hela sanningen, men det är basically så det fungerar. Du kan läsa på mer om statiskt vs dynamiskt typade språk här.

Syntax i Python

I följande avsnitt står det hur funktioner, if-satser, loopar, och variabler skrivs i Python.

Många exempel nedan innehåller text skriven med “#” framför, detta är kommentarer och gör ingenting i programmet, utan är bara till för att lägga till information utöver koden.

Vi använder även >> för att indikera vad som skulle skrivas ut i terminalen ifall programmet skulle köras, det är ingenting som faktiskt används i språket utan finns bara där som förtydligande.

Kod skriver man även på *engelska*, att skriva kod på t.ex svenska är mycket mycket cursed och kommer anmälas till disciplinnämnden. Kommentarer o.s.v försöker man också hålla på engelska, men vi är lite liberala i detta fall för att överdriva med förtydligandet.

Variabler

Variabler har ingen strikt typning i Python, alltså att variabler kan anta nästan vilket värde som helst. De definieras på följande sätt:

Python

```python
x = 1 # int = integer = heltalsvärde
y = 2.5 # float = flyttalsvärde = inte heltal
a = True # bool = antingen sant eller falskt
b = False
text = "This is some text" # str = string = text

numberList = [1,2,3,4,"text"] # list = lista av värden

dictionary = { # dict = lista av 'nycklar' som har egna värden
  "MafU": "D-Tour",
  "LiU": "universitet",
  "trivial": [1,2,3,4]
}

print(numberList[0])
>> 1
print(numberList[5])
>> "Text"
print(text)
>> "This is some text"
print(dictionary['MafU'])
>> "D-Tour"
```

If-satser

If-satser är ett sätt att styra vilka delar av programmet som körs, se nedan:

```python
x = 2

if x == 2:
    print("X is 2!")
else:
    print("X is not 2")

>> X is 2!
```

Funktioner

Funktioner är block av kod som kan anropas, vilket underlättar mycket då man skriver samma sak flera gånger.

Istället för att koden ser ut såhär

```python
def main():
      x = 3
      y = 5
      z = x + y
      print(z)
# Vi förklarar vad __name__ == "__main__" innebär
# lite längre ner i dokumentet
if __name__ == "__main__":
      main()

>> 8
```

så kan koden se ut såhär

```python
def add(argument1, argument2):
      '''Dokumentation för att klargöra vad funktionen gör'''
      print(argument1 + argument2)

def main():
      add(3, 5)

if __name__ == "__main__":
      main()

>> 8
```

Där bägge dessa filer gör exakt samma sak, men den andra är mycket mer läsbar. Läsbarhet är något man strävar efter med sin kod, dels för sin egen del och dels för andra som läser den. Utöver att ha så kallad ‘självdokumenterande’-kod så kan man lägga till mer tydliggörande dokumentation, men detta kan vi skita i för det är onödigt i denna workshop.

Mer om funktioner

Funktioner kan även användas för att *returnera* olika värden, vilket innebär att man kan tilldela värden på variabler direkt med funktioner

Python

```
def add(a, b):
  c = a + b
  return c

def main():
  d = add(3,5)
  print(d)

if __name__ == "__main__":
  main()

>> 8
```

Loopar

Loopar är kod som körs flera gånger, och kan definieras huvudsakligen på tre olika sätt:

Python

```
# Standard for-loop (i kommer gå från 1 till och med 9)
for i in range(1,10):
    print(i)

>> 1
>> 2
>> ...
>> 9

# Foreach-loop (kommer gå igenom varje element i listan nedan)
myList = ['a','b','c']
for letter in myList:
    print(letter)

>> a
>> b
>> c

# While-loop (kommer printa 1,2,3)
x = 0
while (x < 3):
    x += 1 # x++ finns inte i Python
    print(x)

>> 1
>> 2
>> 3
```

C. Komma igång med Python

För att komma igång kan du börja med att skapa och köra ett första Python-program!

Nu ska du få testa skriva Pythonkod! Du kan kopiera in följande i din nya fil (eller skriva något eget):

```python
def main():
    print("Hello world!")

# Standard syntax för att köra main-funktionen
if __name__ == "__main__":
    main()
```

__name__ == “__main__” används för att typ “skydda” dokumentet ifall det skulle *importeras* (kan du läsa mer om här). Detta är dock inget som vi behöver tänka på i denna workshop.

För att köra koden går du tillbaka till terminalen och skriver in följande:

```python
python start.py
```

Du borde nu fått ut meddelandet “Hello world!” i terminalen, bra jobbat!

Du kan nu testa koda själv, se nästa kapitel för att testa på hur man gör lite olika saker i Python.

D. uppgifter

Nedan beskrivs ett antal problem, dessa behövs inte göras i någon speciell ordning. Försök att lösa så många du kan!

1. Skriv en funktion som summerar alla jämna tal från 0-99

Skapa en loop som räknar upp från 0 upp till 99 (ej talet 100) och summera alla jämna tal till en variabel. Printa sedan denna variabel, resultatet bör vara 2450.

Tips: använd modulo i kombination med en if-sats för att hitta jämna tal. Modulo kan ses som resten man får från heltalsdivision.

```python
print(10//3) # // används för heltalsdivison
>> 3
print(10%3) # % används för modulo
>> 1
```

2. Skriv en funktion som hittar största talet i en lista av siffror

Skriv en funktion som tar en lista med siffror som argument och returnerar det största talet i denna lista. Exempel:

```python
numbers = [1,3,5,9,2]

largest = find_max(numbers)

# f:et står för 'format' och innebär att istället för att
# skriva ut 'largest' så använder vi värdet från variabeln istället
print(f"The largest number is: {largest}")

>> The largest number is: 9
```

3. Skriv en funktion som rekursivt räknar ut fakulteten av ett tal

Skriv en funktion som beräknar fakulteten av ett godtyckligt tal som skickas som argument till denna funktion.

Denna funktion ska vara rekursiv, det vill säga att den anropar sig själv. Viktigaste med rekursiva funktioner är så kallade ‘basfall’, alltså när funktionen ska sluta att kalla på sig själv.

```python
def factorial(n):
	if n == 0:
		return 1
	...
```

Förtydligande/repetition, fakultet brukar skrivas med utropstecken i matematik. Ex:

- 5! = 5*4*3*2*1 = 120

```python
n = 5
factorial(n)

>> 120
```

4. Skriv en funktion som genererar Fibonaccis talserie

Skriv en funktion som tar in ett argument, n, och returner tal i Fibonaccis talserie upp till och med talet n som en lista. Fibonaccis talserie börjar med 0, 1 och summerar två efterföljande tal för att få nästa tal.

```python
n = 7
fibonacci(n)

>> 0, 1, 1, 2, 3, 5
```

5. Skriv en funktion som vänder på en mening

Skriv en funktion som tar in en mening och skriver ut orden i meningen i omvänd ordning.

```python
string = "MafU älskar programmering"
reverse(string)

>> "programmering älskar MafU"
```

6. Skriv en funktion som räknar antalet av varje karaktär i en sträng

Skriv en funktion som tar in en sträng och skriver ut antalet gånger varje karaktär dyker upp i strängen. Till din hjälp bör du använda dig av en dictionary.

```python
string = "Mississippi"
letter_counter(string)

>> {'m': 1, 'i': 4, 's': 4, 'p': 2}
```

7. Skriv en funktion som kör gissningslek

Skriv en funktion som tar fram ett slumpat heltalsvärde, och avslutas först när användaren gissar rätt tal.

tips: använd en while-loop med if-satser innuti

```python
# I python används input('Här skriver man vad som skrivs ut i terminalen')
för att låta användaren mata in ett värde

guess = input('Enter your guess: ')
>> Enter your guess:
```

8. Skriv en funktion som jämför ett argument med en lista

```python
argument = 5
list_of_numbers = [1,2,3,4] # ordet 'list' är redan taget av python så man får vara lite kreativ
compare(argument, lista)
>> False
```

E. Inbyggda funktioner

I de flesta programmeringsspråk finns redan det allra mesta implementerat, så man slipper uppfinna hjulet på nytt. T.ex tänk om det fanns ett sätt att ‘mappa’ en funktion till alla värden i en lista?

```python
numbers = [1,2,3,4]
def squared(number):
  return number**2 # ** innebär upphöjt till
squared_numbers = map(squared, numbers)
# map returnerar dock en 'iterator', något som man kan stega över
# vilket innebär att vi måste konvertera det till en lista först
# innan vi kan göra något vettigt med den

print(list(map(squared, numbers)))
>> [1,4,9,16]
```

Det finns ett x antal olika användbara liknande funktioner, så om du tänker att det hade varit vettigt med en viss grej, googla och kolla om det redan finns en bra lösning!

Nedanför introducerar vi lambda, en så kallad anonym funktion, som används för att skapa funktioner (precis som med def) men i syfte för *väldigt* kortfattade funktioner och fungerar på följande sätt: lambda argument: gör något med argumentet. Vi tar förra exemplet, gör precis samma sak fast på mindre rader kod.

```python
numbers = [1,2,3,4]
squared_numbers = map(lambda x: x**2, numbers)
print(list(squared_numbers))
>> [1,4,9,16]

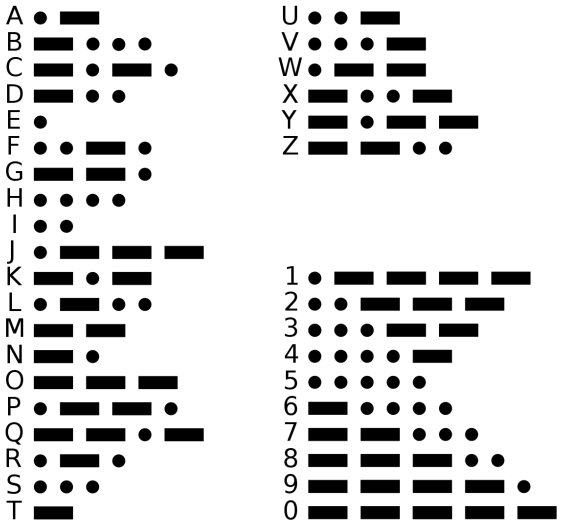
# här är lambda x: x**2 precis samma sak som funktionen squared som
# definierade i förra exemplet
```

I python finns det en dum mängd olika sådana hära inbyggda funktioner, som du kan hitta här

F. Svåra uppgifter

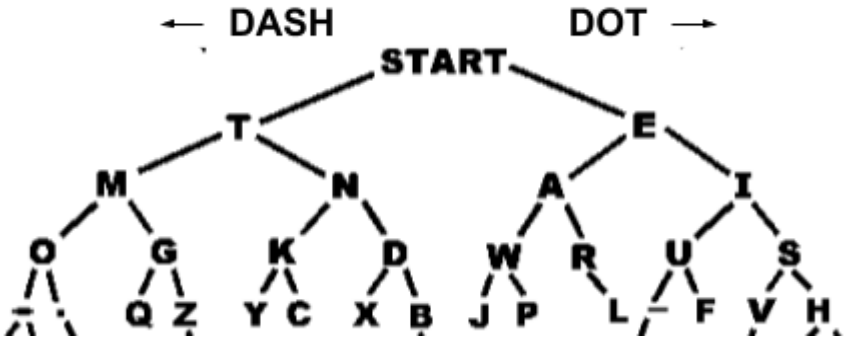
1. Skapa en funktion som översätter morsekod till bokstäver

Skapa en funktion som översätter morsekod i en sträng till bokstäver. Morsekod definieras på följande sätt där varje bokstav är uppbyggd av “dots” (.) och “dashes” (-):



För enkelhets skull kan vi säga att mellanrum i Morsekaraktärer är ett mellanslag “ “ och mellanrum i Morseord är mellanslag, slash, mellanslag “ / “.

Detta kan representeras med en trädstruktur enligt nedan, där ett steg ner till höger motsvarar “dot” och ett steg ner till vänster motsvarar “dash”.



T.ex. morsekoden “-..” (bokstaven “D”) motsvarar ett steg ner till vänster följt av två steg ner till höger, dvs att gå från START -> T -> N -> D.

Ett komplett träd går att representera som en lista i kod, förutsatt att man använder korrekta funktioner för att komma åt element i trädet.

Försök avkoda följande meddelande med ditt program:
“-.-. --- -. --. .-. .- - ..- .-.. .- - .. --- -. ... / -.-- --- ..- / ... --- .-.. ...- . -.. / .. -”
Du kan utgå från nedanstående kod för att komma igång, [länk till kodskelett med koden nedan].

Python
```
import math

charTree = [
  '-', 'T','E','M','N','A','I','O','G','K','D',
  'W','R','U','S',' ',' ','Q','Z','Y','C','X','B',
  'J','P',' ','L',' ','F','V','H'
]


def left(i):
    """ Returns index of left child"""
    return 2*i + 1


def right(i):
    """ Returns index of right child """
    return 2*i + 2


def parent(i):
    """ Returns index of parent """
    return math.floor((i-1)/2)


def isLeaf(i):
    """ Returns True if i is leaf, otherwise False"""
    if (i >= len(charTree) or charTree[i] == ' '):
        return True
    return False
```

Intressanta länkar till Morseproblemet

- Mer info om att spara träd som listor (ignorera pop-up om betyg): https://www.ida.liu.se/opendsa/Books/TDDD86F22/html/CompleteTree.html
- Morsekodsgenerator och avkodare: https://morsecode.world/international/translator.html

Fler svårare problem

Nedan finns några länkar till andra svåra problem, dessa finns dock mindre hjälp tillgänglig för, men ni kan alltid fråga oss så försöker vi lösa det åt er!

- Laborationer från första programmeringskurs: https://www.ida.liu.se/~TDDE23/planering.shtml
- Avancerade problem (ett av dessa är Morsekod-exemplet): https://www.codecademy.com/resources/blog/advanced-python-code-challenges/